

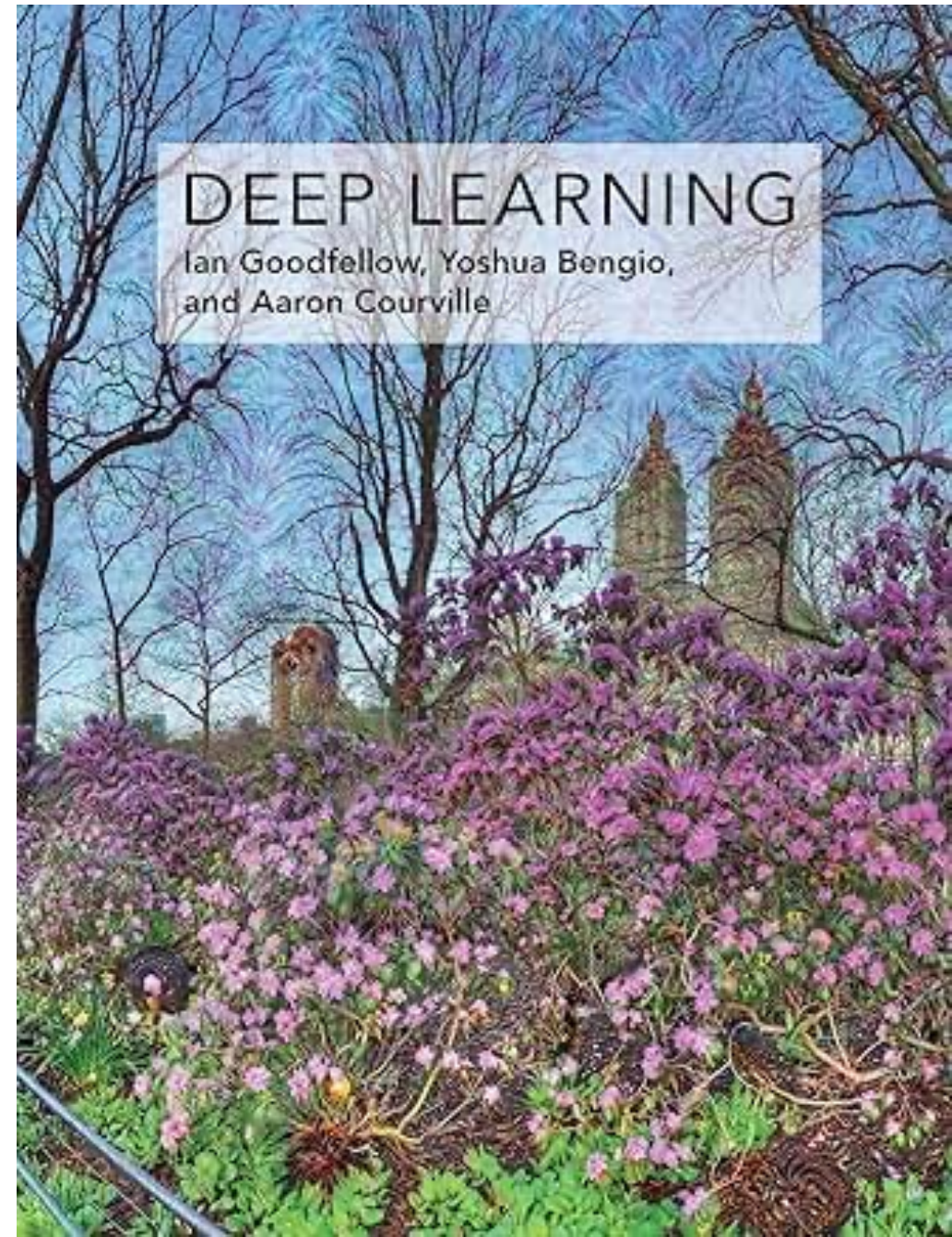


CSCI 6032: Lecture 4: Agents

Instructor: Dr. David A. G. Harrison

Things I want to cover today...

- **Agents**
 - Setting up GitHub Copilot CLI
 - What makes a CLI agent different from a chat window
 - Tools, permissions, and the agent loop
 - Skills: packaging repeatable workflows
- **Do it securely.**



Office Hours

207 Weir Hall

Monday	11:00 AM-12:00 PM
Wednesday	1:00-2:00 PM



HW1 (Reminder)

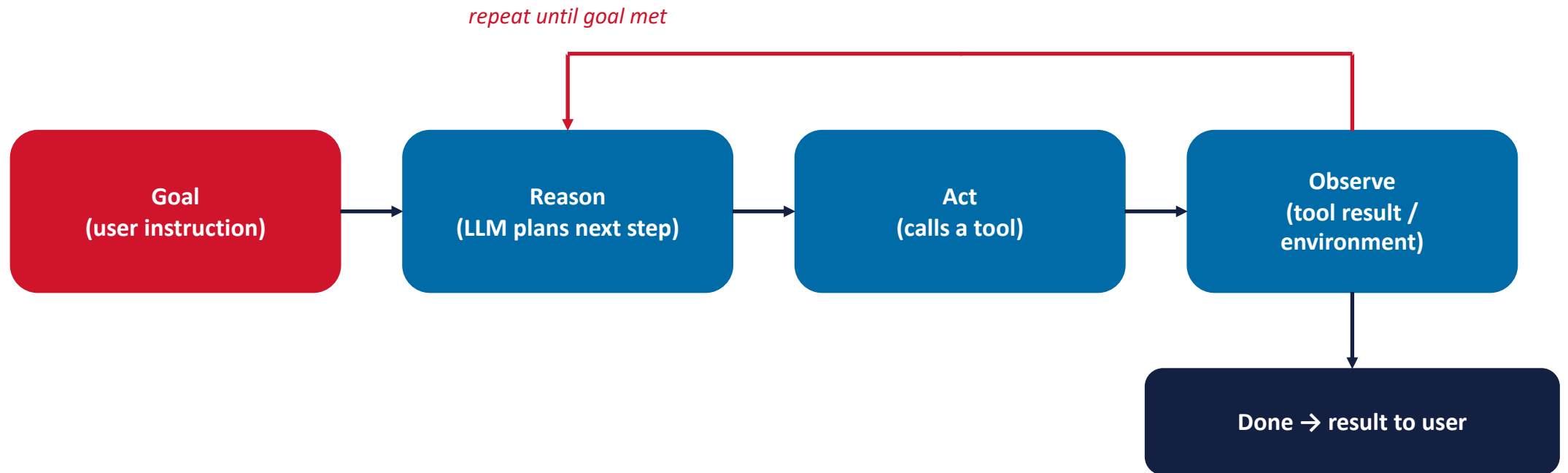
On github already...

Due today at 11pm

Submit as notebook via blackboard.

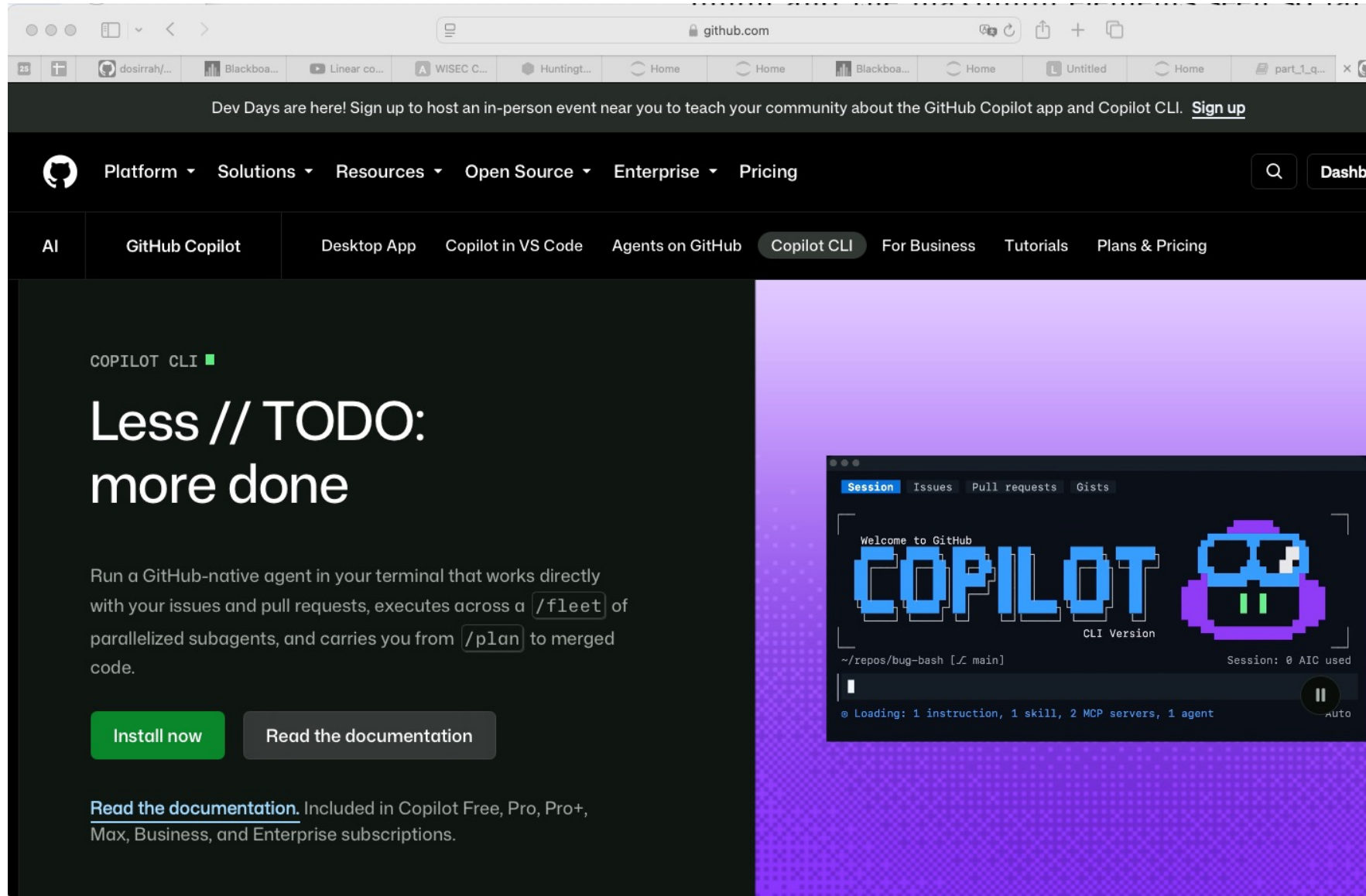
What Is an Agent?

A program that loops: reason about the goal, act in the world, observe what happened — and repeat until the goal is done.



Agent = model (reasoning) + tools (ability to act) + a loop, operating with some autonomy toward a goal.

Agents: GitHub Copilot



Dev Days are here! Sign up to host an in-person event near you to teach your community about the GitHub Copilot app and Copilot CLI. [Sign up](#)

Platform Solutions Resources Open Source Enterprise Pricing

AI GitHub Copilot Desktop App Copilot in VS Code Agents on GitHub Copilot CLI For Business Tutorials Plans & Pricing

COPILOT CLI

Less // TODO: more done

Run a GitHub-native agent in your terminal that works directly with your issues and pull requests, executes across a `/fleet` of parallelized subagents, and carries you from `/plan` to merged code.

[Install now](#) [Read the documentation](#)

[Read the documentation.](#) Included in Copilot Free, Pro, Pro+, Max, Business, and Enterprise subscriptions.



Ole Miss

Agents: GitHub Copilot

<https://github.com/settings/education/benefits>



David Harrison (dosirrah)

Your personal account

 Menu

 Switch context ▾

**Free Student
edition.**

GitHub Education

Free GitHub developer resources for students and teachers

Get Copilot for free, 180 monthly Codespaces hours for cloud coding, unlimited private repositories with GitHub Pro or Team, and dozens of premium tools in the Student Developer Pack.

GitHub Copilot

Command-Line Interface

- **Runs in terminal (shell)**
- **Can install in linux, windows, or macOS.**

Use one of these commands:

- **Cross-platform (npm)**

Prerequisite: Node.js 22 or later.

Bash

```
npm install -g @github/copilot
```

- **Windows (WinGet)**

Bash

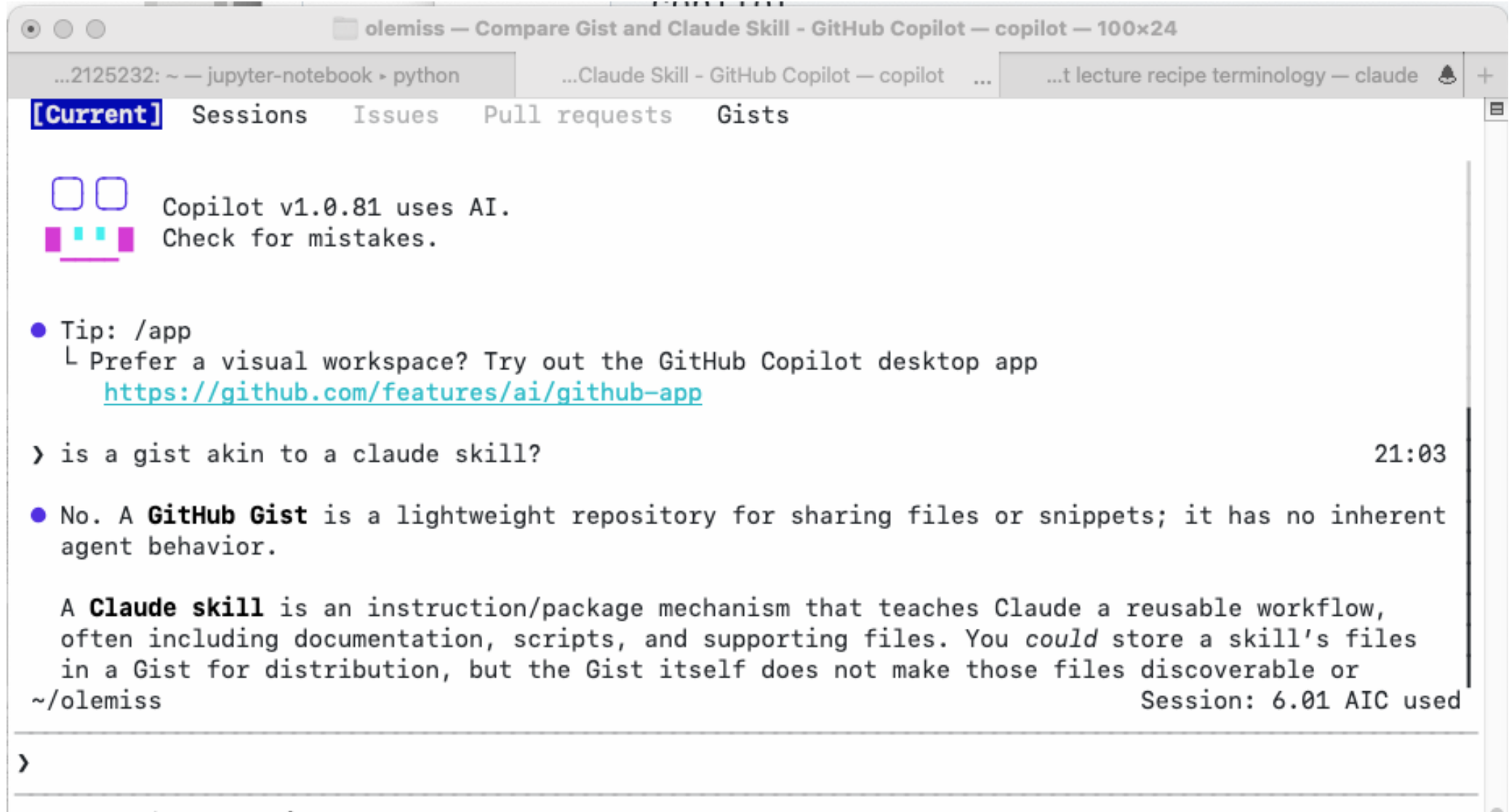
```
winget install GitHub.Copilot
```

- **macOS/Linux (Homebrew)**

Bash

```
brew install --cask copilot-cli
```


GitHub Copilot



The screenshot shows a GitHub Copilot interface with a terminal window. The terminal title is "olemiss — Compare Gist and Claude Skill - GitHub Copilot — copilot — 100x24". The terminal content includes a message about Copilot v1.0.81 using AI, a tip about the GitHub Copilot desktop app, and a comparison between GitHub Gists and Claude Skills. The terminal also shows a session duration of 21:03 and a session ID of 6.01 AIC used.

```
olemiss — Compare Gist and Claude Skill - GitHub Copilot — copilot — 100x24
...2125232: ~ — jupyter-notebook > python
...Claude Skill - GitHub Copilot — copilot
...t lecture recipe terminology — claude
[Current] Sessions Issues Pull requests Gists

Copilot v1.0.81 uses AI.
Check for mistakes.

• Tip: /app
  ↳ Prefer a visual workspace? Try out the GitHub Copilot desktop app
  https://github.com/features/ai/github-app

> is a gist akin to a claude skill? 21:03

• No. A GitHub Gist is a lightweight repository for sharing files or snippets; it has no inherent agent behavior.

A Claude skill is an instruction/package mechanism that teaches Claude a reusable workflow, often including documentation, scripts, and supporting files. You could store a skill's files in a Gist for distribution, but the Gist itself does not make those files discoverable or
~/olemiss Session: 6.01 AIC used

>
```

What Makes a CLI Agent Different from a Chat Window?

A chat window answers questions; a CLI agent takes action

- Chat: you copy code out, run it, and paste results back in
- CLI agent: reads, edits, and runs code directly in your project

It can take multiple steps on its own

- Plans, calls tools, checks results, and continues — without a relay

It needs permission boundaries

- Because it acts instead of just talking, what it's allowed to do matters

Tools

A tool is a specific capability the agent can invoke

- Read / write / edit files
- Run shell commands
- Search the web, browse a page
- Query an API, a database, a service

The agent decides which tool to call and with what arguments

Tool results feed back into its reasoning for the next step

More tools = more capability, but also more surface area for mistakes

Permissions

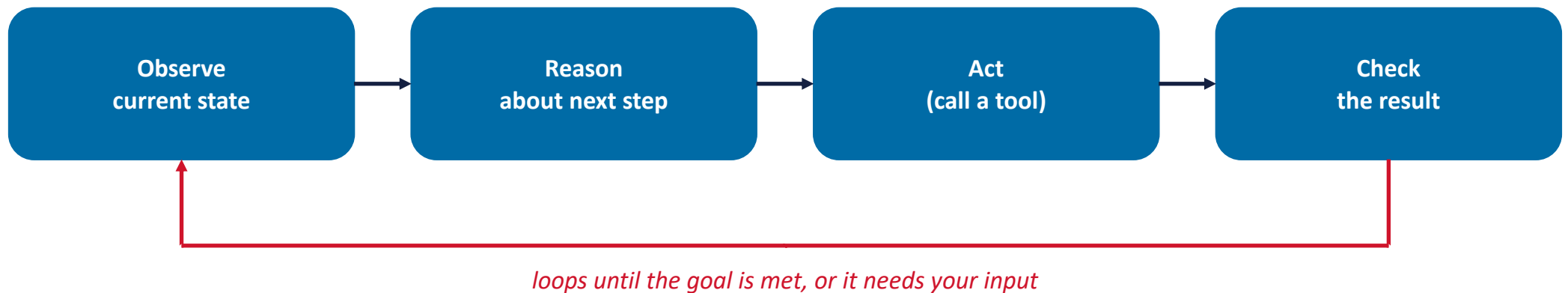
Every tool call is a real action — permissions decide which ones need a human first

- **Auto-approved**
 - Safe, reversible actions (e.g., reading a file, listing a directory)
- **Ask first**
 - Risky or hard-to-reverse actions (e.g., editing files, running a command, pushing to git)
- **Denied**
 - Actions explicitly out of bounds for the task

Good permission design assumes the agent will eventually be wrong about something

- You're bounding the blast radius of a mistake, not just approving good behavior

The Agent Loop



- Each iteration, the agent decides: keep going, ask the user, or stop.
- It stops when the goal is met, it hits a permission boundary, or it's stuck and needs help.
- This is what separates an agent from a single autocomplete suggestion.

Skills: Packaging Repeatable Workflows

A skill is a saved, reusable recipe for a task you do more than once

- Instructions the agent follows step by step (e.g., “post a lecture”, “grade a homework”)
- Can bundle scripts, templates, and checklists alongside the instructions

Why bother?

- Consistency — the same steps happen the same way every time
- Less re-explaining — you don't retype the process in every session
- Shareable — teammates (or future you) can reuse the same workflow

Creating a Skill

A skill is just a folder with instructions the agent can discover and load

- A short description of when to use it (its trigger)
- Step-by-step instructions written in plain language
- Optional helper scripts or reference files it can call

The agent decides on its own when a skill applies

- You describe the task; if it matches a skill's description, the agent invokes it

Good skills are narrow and explicit

- One clear workflow per skill beats one skill that tries to do everything